

Министерство науки и высшего образования РФ
федеральное государственное автономное
образовательное учреждение высшего образования
«Омский государственный технический университет»

Факультет информационных технологий и компьютерных систем
Кафедра «Прикладная математика и фундаментальная информатика»

Расчетно-графическая работа
по дисциплине **Машинное обучение и большие данные**
Тема: Разработка Web-приложения (дашборда)
для инференса моделей ML и анализа данных

Студента	Колодницкого Ильи Михайловича
Курс	<u>2</u> Группа <u>ФИТ-232</u>
Направление	<u>02.03.02 Фундаментальная информатика и</u> <u>информационные технологии</u>
Руководитель	<u>старший преподаватель</u> <u>Шарун Иван Владимирович</u>
Выполнил	<u>7.6.25</u> 
Проверил	

Омск 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Описание выбранных моделей	4
1.1 К ближайших соседей	4
1.2 Сериализация	4
1.3 Bagging Regressor, Gradient Boosting Regressor, Stacking Regressor	6
2 Процесс разработки дашборда	6
2.1 Разработка страницы с информацией об авторе.....	7
2.2 Разработка страницы с информацией о датасете	7
2.3 Разработка страницы с визуализацией данных	8
2.4 Разработка страницы для получения инференса	9
ЗАКЛЮЧЕНИЕ.....	13
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	13
ПРИЛОЖЕНИЕ А.....	15
ПРИЛОЖЕНИЕ Б (обязательное)	20

ВВЕДЕНИЕ

Актуальность темы данной РГР заключается в практическом применении методов машинного обучения и визуальной аналитики через разработку интерактивного Web-приложения. Работа направлена на решение задачи диагностики сердечно-сосудистых заболеваний на основе медицинских данных — посредством построения дашборда с возможностью инференса моделей.

В процессе выполнения были изучены и использованы следующие инструменты:

- **TensorFlow/Keras** — для реализации нейросетевой архитектуры,
- **pickle** — для сериализации и сохранения обученных моделей и препроцессоров,
- **streamlit** — для построения веб-приложения, обеспечивающего загрузку данных, визуализацию и проведение предсказаний,
- а также библиотеки **seaborn** и **matplotlib** — для анализа и визуализации данных.

Целью РГР является создание Web-приложения, которое предоставляет пользователю удобный интерфейс для работы с моделями машинного обучения: от анализа данных до получения предсказаний.

Для реализации поставленной цели были решены следующие задачи:

1. Сформировать и обучить несколько моделей машинного обучения.
2. Сохранить обученные модели и предусмотреть возможность их повторного использования.
3. Разработать Web-интерфейс с реализацией предсказания как на загружаемых данных, так и в режиме ручного ввода.
4. Обеспечить визуальный анализ исходных и предсказанных данных с использованием графиков и тепловых карт.

1 Описание выбранных моделей

1.1 К ближайших соседей

Использовался алгоритм ближайших соседей с евклидовой метрикой (`metric='euclidean'`), числом соседей `n_neighbors=3` и взвешиванием по расстоянию (`weights='distance'`). Алгоритм хорошо себя показал в ситуациях, когда классы были хорошо разделены в пространстве признаков. В качестве ускорения был выбран алгоритм поиска `kd_tree`, а `leaf_size` установлен в 10. Данная модель проста в реализации и обеспечивает интерпретируемость результатов.

1.2 Сериализация

Для сериализации моделей, обученных с использованием библиотеки *scikit-learn*, применялась библиотека *pickle* и её метод *dump*, позволяющий сохранять модель в файл с расширением *.pkl*. С помощью той же библиотеки осуществлялась десериализация — извлечение моделей из файлов и их интеграция.

На рисунке 1 представлена реализация процесса сохранения всех моделей [1].

```

# 2. Gradient Boosting
gb_pipe = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', GradientBoostingClassifier(
        ccp_alpha=0.01471511413755825,
        learning_rate=0.5339651291062001,
        max_depth=1,
        max_leaf_nodes=15,
        min_samples_leaf=12,
        min_samples_split=20,
        n_estimators=99,
        random_state=42))
])
gb_pipe.fit(X, y)
with open('model_gb.pkl', 'wb') as f:
    pickle.dump(gb_pipe, f)

# 3. CatBoost
catboost_model = CatBoostClassifier(
    depth=9,
    grow_policy='Depthwise',
    iterations=57,
    learning_rate=0.03612670446188109,
    cat_features=[features.index(col) for col in cat_features],
    silent=True,
    random_seed=42
)
X_catboost = X.copy()
for col in cat_features:
    X_catboost[col] = X_catboost[col].astype(str)

catboost_model.fit(X_catboost, y)
with open('model_catboost.pkl', 'wb') as f:
    pickle.dump(catboost_model, f)

# 4. Random Forest
rf_pipe = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(
        max_features=0.7043559771674565,
        max_samples=0.9705153393013579,
        n_estimators=92,
        random_state=42))
])
rf_pipe.fit(X, y)
with open('model_rf.pkl', 'wb') as f:
    pickle.dump(rf_pipe, f)

```

Рисунок 1 – Сохранение моделей через библиотеку pickle

1.3 Bagging Regressor, Gradient Boosting Regressor, Stacking Regressor

Дополнительно были выбраны ансамблевые модели классификации: **Random Forest Classifier**, **Gradient Boosting Classifier** и **Stacking Classifier** из библиотеки `scikit-learn`. Их выбор обоснован результатами практического тестирования, проведённого в рамках выполнения лабораторной работы [2].

- **Random Forest Classifier** позволяет достичь высокой устойчивости к переобучению за счёт построения множества независимых деревьев решений на случайных подвыборках признаков и данных. Модель показала хорошую обобщающую способность и стабильность на различных кросс-валидационных фолдах. Применялись параметры: `n_estimators=92`, `max_features=0.704`, `max_samples=0.971`.
- **Gradient Boosting Classifier** оказался эффективным для задачи бинарной классификации, особенно на сложных паттернах с нелинейными зависимостями. Он последовательно обучает слабые модели (деревья) на ошибках предыдущих, повышая общую точность предсказания. Модель использовала такие параметры, как `learning_rate=0.533`, `n_estimators=99`, `max_depth=1`, `min_samples_split=20`, `min_samples_leaf=12`, `ccp_alpha=0.0147`.
- **Stacking Classifier** применялся для объединения предсказаний нескольких базовых моделей: `DecisionTreeClassifier`, `SVC`, `KNeighborsClassifier`, `GaussianNB`. Использование стекинга позволило интегрировать сильные стороны различных алгоритмов и повысить надёжность финального прогноза. Стекинг оказался особенно полезным на нерегулярных подвыборках и при высокой дисперсии между моделями.

2 Процесс разработки дашборда

В ходе работы было принято решение реализовывать дашборд поэтапно. На первом этапе была разработана его архитектура, определяющая структуру и основные компоненты интерфейса. На следующем этапе была выполнена интеграция обученных моделей в веб-приложение, что позволило обеспечить функциональность инференса непосредственно через дашборд [3].

2.1 Разработка страницы с информацией об авторе

Для начала была реализована страница с информацией об авторе. На данной странице представлены ФИО, учебная группа, фотография и тема РГР. Также размещены четыре ссылки для перехода на остальные страницы дашборда.

Для отображения информации использовались функции *title*, *markdown* и *image*, предоставляемые библиотекой *Streamlit*.

Пример кода, реализующего страницу с данными об авторе, показан на рисунке 2.

```
def info_page():
    st.title('Информация')

    st.markdown("ФИО: Колодницкий Илья Михайлович")
    st.markdown("Группа: ФИТ-232")

    st.image("therock.jpg", width=900, caption="Я на пляже")

    st.subheader("Тема РГР: :blue[«Разработка Web-приложения (дашборда) для инференса (вывода) моделей ML и анализа данных»] :sunglasses:")
```

Рисунок 2 – Код для создания страницы об авторе

2.2 Разработка страницы с информацией о датасете

Далее я написал код, реализующий страницу с описанием датасета и особенностями его предобработки. Для структурирования информации и удобства взаимодействия с пользователем были использованы методы *cache_data* и *markdown*

На рисунке 3 показан фрагмент кода, демонстрирующий создание страницы об описании датасета.

```

def data_page():
    st.title("Информация о наборе данных")
    data = load_data()

    st.header("Описание предметной области")
    st.write("""
    Цель данного проекта – предсказать наличие сердечного заболевания на основе медицинских показателей.
    Сердечно-сосудистые заболевания являются одной из ведущих причин смертности во всём мире.
    Этот датасет собран из клинических данных пациентов и используется для построения моделей машинного обучения,
    способных выявить возможное заболевание на ранних этапах.
    """)

    st.header("Описание признаков")
    st.write("""
    - **age**: возраст пациента
    - **sex**: пол
    - **cp**: тип боли в груди
    - **trestbps**: артериальное давление в покое
    - **chol**: уровень холестерина
    - **fbs**: уровень сахара натощак
    - **restecg**: результаты ЭКГ
    - **thalach**: максимальный пульс
    - **exang**: стенокардия при физической нагрузке
    - **oldpeak**: депрессия ST
    - **slope**: наклон сегмента ST
    - **ca**: количество крупных сосудов
    - **thal**: результат теста таллия
    - **target (или num)**: наличие сердечного заболевания
    """)

    st.header("Особенности предобработки данных")
    st.write("""
    - Удаление/замена пропущенных значений
    - Нормализация числовых признаков с помощью StandardScaler
    - Кодирование категориальных признаков с помощью OneHotEncoder/LabelEncoder
    - Балансировка классов (например, через SMOTE)
    - Разделение на обучающую и тестовую выборки
    """)

    st.header("Разведочный анализ данных (EDA)")
    st.subheader("Распределение целевой переменной")
    fig1, ax1 = plt.subplots()
    sns.countplot(x='num', data=data, ax=ax1)
    ax1.set_title("Наличие сердечного заболевания")
    st.pyplot(fig1)

```

Рисунок 3 – Код для создания страницы о датасете

2.3 Разработка страницы с визуализацией данных

На данной странице пользователь может увидеть графики распределения таргета, непрерывных и категориальных признаков. Для этого использовались такие виды графиков как *histplot*, *heatmap*, *boxplot*, *scatterplot*. На рисунке 4 представлена часть реализации разработки страницы с графиками.

```

def graphics_page():
    st.title("Визуализация данных")
    data = load_data()

    st.header("1. Распределение целевой переменной")
    fig, ax = plt.subplots()
    sns.countplot(x='num', data=data, ax=ax)
    ax.set_title("Распределение по наличию сердечного заболевания")
    st.pyplot(fig)

    st.header("2. Корреляционная матрица")
    fig, ax = plt.subplots(figsize=(10, 7))
    sns.heatmap(data.corr(), annot=True, fmt=".2f", cmap="coolwarm", ax=ax)
    st.pyplot(fig)

    st.header("3. Распределение возраста")
    fig, ax = plt.subplots()
    sns.histplot(data['age'], bins=20, kde=True, ax=ax)
    st.pyplot(fig)

    st.header("4. Возраст vs Максимальный пульс")
    fig, ax = plt.subplots()
    sns.scatterplot(x='age', y='thalach', hue='num', data=data, ax=ax)
    st.pyplot(fig)

    st.header("5. Вохplot уровня холестерина")
    fig, ax = plt.subplots()
    sns.boxplot(x='num', y='chol', data=data, ax=ax)
    st.pyplot(fig)

```

Рисунок 4 – Составление графиков

2.4 Разработка страницы для получения инференса

На данной странице пользователь может получить предсказание, сформированное на основе работы ранее описанных моделей, используя введённые данные. Для этого предлагается загрузить собственный датасет.

На рисунке 5 приведён пример кода, отвечающего за загрузку CSV-файла.

```

col1, col2 = st.columns(2)
with col1:
    age = st.number_input("Возраст", 1, 120, 50)
    sex = st.radio("Пол", ["Мужской", "Женский"])
    cp = st.selectbox("Тип боли", ["Типичная", "Атипичная", "Неангинозная", "Бессимптомно"])
    trestbps = st.number_input("Давление (мм рт.ст.)", 80, 200, 120)
    chol = st.number_input("Холестерин (мг/дл)", 100, 600, 200)
    fbs = st.radio("Сахар > 120", ["Да", "Нет"])

with col2:
    restecg = st.selectbox("ЭКГ", ["Норма", "ST-T", "Гипертрофия"])
    thalach = st.number_input("Макс. пульс", 60, 220, 150)
    exang = st.radio("Стенокардия при нагрузке", ["Да", "Нет"])
    oldpeak = st.number_input("Oldpeak", 0.0, 10.0, 1.0, 0.1)
    slope = st.selectbox("Наклон ST", ["Восходящий", "Плоский", "Нисходящий"])
    ca = st.number_input("Сосуды", 0, 3, 0)
    thal = st.selectbox("Таллий", ["Норма", "Фиксированный", "Обратимый"])

sex = 1 if sex == "Мужской" else 0
cp = {"Типичная": 1, "Атипичная": 2, "Неангинозная": 3, "Бессимптомно": 4}[cp]
fbs = 1 if fbs == "Да" else 0
restecg = {"Норма": 0, "ST-T": 1, "Гипертрофия": 2}[restecg]
exang = 1 if exang == "Да" else 0
slope = {"Восходящий": 1, "Плоский": 2, "Нисходящий": 3}[slope]
thal = {"Норма": 3, "Фиксированный": 6, "Обратимый": 7}[thal]

input_data = pd.DataFrame([age, sex, cp, trestbps, chol, fbs, restecg,
                             thalach, exang, oldpeak, slope, ca, thal],
                           columns=['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs',
                                    'restecg', 'thalach', 'exang', 'oldpeak',
                                    'slope', 'ca', 'thal'])

```

Рисунок 5 – Код для загрузки датасета

Затем был написан код, отвечающий за загрузку моделей, а также отображение полей для ввода данных пользователем. Пример такого кода приведен на рисунке 6.

```

def predict_page():
    st.title("Предсказание диагноза")
    models = load_models()
    if not models:
        st.error("Модели не загружены")
        return

    model_choice = st.selectbox("Выберите модель", list(models.keys()))

    st.subheader("1. Загрузка файла с данными")
    uploaded_file = st.file_uploader("Загрузите CSV-файл для пакетного предсказания", type=["csv"])
    if uploaded_file:
        try:
            input_df = pd.read_csv(uploaded_file)

            if model_choice == "CatBoost":
                for col in ['cp', 'fbs', 'restecg', 'exang', 'slope', 'thal']:
                    if col in input_df.columns:
                        input_df[col] = input_df[col].astype(str)

            if st.button("Предсказать из файла"):
                if model_choice == "Neural Network":
                    preprocessor, nn_model = models[model_choice]
                    input_transformed = preprocessor.transform(input_df)
                    prediction = (nn_model.predict(input_transformed) > 0.5).astype(int)
                    input_df["Предсказание"] = prediction
                else:
                    model = models[model_choice]
                    prediction = model.predict(input_df)
                    input_df["Предсказание"] = prediction

                st.subheader("Результаты предсказания")
                st.dataframe(input_df)
                return

        except Exception as e:
            st.error(f"Ошибка при обработке файла: {e}")

```

Рисунок 6 – Код для записи данных пользователем

Далее пользователь может получить предсказание от моделей, нажав на кнопку “Получить прогноз срабатывания датчика”. После ее нажатия выдаётся ответ. Ниже на рисунке 7 представлен код для получения предсказания.

```

if st.button("Предсказать"):
    if model_choice == "Neural Network":
        try:
            preprocessor, nn_model = models[model_choice]
            input_transformed = preprocessor.transform(input_data)
            prediction = (nn_model.predict(input_transformed) > 0.5).astype(int)
            proba = [1 - prediction[0][0], prediction[0][0]]
        except Exception as e:
            st.error(f"Ошибка при обработке нейросети: {e}")
            return
    elif model_choice == "CatBoost":
        model = models[model_choice]
        input_df = pd.DataFrame(input_data, columns=[
            'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
            'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal'
        ])
        for col in ['cp', 'fbs', 'restecg', 'exang', 'slope', 'thal']:
            input_df[col] = input_df[col].astype(str)

        prediction = model.predict(input_df)
        proba = model.predict_proba(input_df)[0]
    else:
        model = models[model_choice]
        prediction = model.predict(input_data)
        proba = model.predict_proba(input_data)[0]

st.subheader("Результат")
if prediction[0] == 1:
    st.error(f"Риск заболевания: {proba[1] * 100:.2f}%")
else:
    st.success(f"Вероятность отсутствия болезни: {proba[0] * 100:.2f}%")

```

Рисунок 7 – Код для получения предсказания

ЗАКЛЮЧЕНИЕ

В рамках работы была изучена библиотека *Streamlit*, предназначенная для создания интерактивных веб-приложений на языке программирования *Python*.

Было реализовано веб-приложение (дашборд), включающее четыре основные страницы:

- страница с информацией об авторе и теме проекта,
- страница с описанием датасета и результатами разведочного анализа (EDA),
- страница с визуализацией ключевых характеристик данных,
- страница для получения инференса моделей машинного обучения, как в пакетном, так и в ручном режиме.

В проекте реализована загрузка и применение различных моделей (KNN, Random Forest, Gradient Boosting, CatBoost, Stacking и нейросеть), а также механизм выбора модели пользователем. Кроме того, предусмотрена предобработка данных, визуализация ключевых статистик и возможность загрузки CSV-файлов для пакетной обработки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Как освоить Streamlit для DS [электронный ресурс]: Режим доступа: <https://habr.com/ru/articles/824064/> – 06.06.25.
- 2) A faster way to build and share data apps [электронный ресурс]: Режим доступа: <https://streamlit.io/> – 06.06.25.
- 3) scikit-learn documentation [электронный ресурс]: Режим доступа: <https://scikit-learn.org/stable/index.html> – 06.06.25.

ПРИЛОЖЕНИЕ А (обязательное)

Скриншоты дашборда

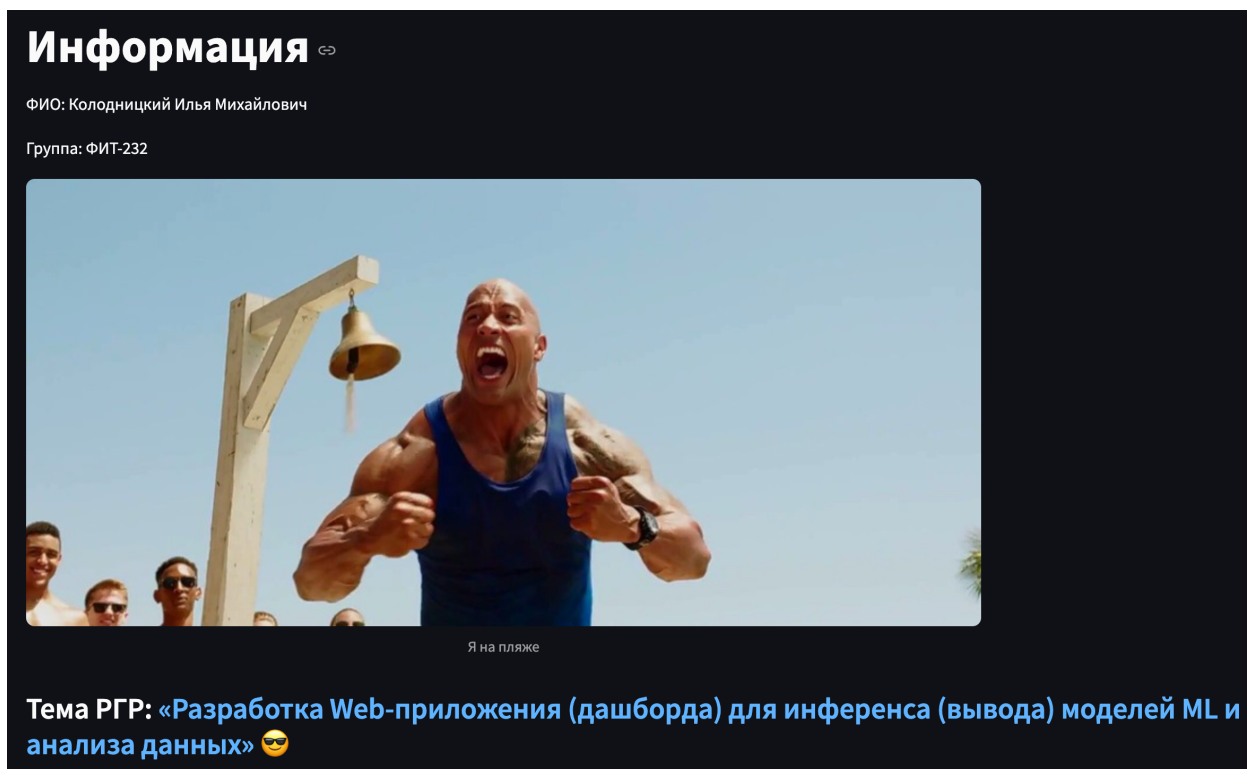


Рисунок А.1 – Страница с информацией об авторе

Описание предметной области

Цель данного проекта — предсказать наличие сердечного заболевания на основе медицинских показателей. Сердечно-сосудистые заболевания являются одной из ведущих причин смертности во всём мире. Этот датасет собран из клинических данных пациентов и используется для построения моделей машинного обучения, способных выявить возможное заболевание на ранних этапах.

Описание признаков

- **age**: возраст пациента
- **sex**: пол
- **cp**: тип боли в груди
- **trestbps**: артериальное давление в покое
- **chol**: уровень холестерина
- **fbs**: уровень сахара натощак
- **restecg**: результаты ЭКГ
- **thalach**: максимальный пульс
- **exang**: стенокардия при физической нагрузке
- **oldpeak**: депрессия ST
- **slope**: наклон сегмента ST
- **ca**: количество крупных сосудов
- **thal**: результат теста таллия
- **target** (или **num**): наличие сердечного заболевания

Особенности предобработки данных

- Удаление/замена пропущенных значений
- Нормализация числовых признаков с помощью `StandardScaler`
- Кодирование категориальных признаков с помощью `OneHotEncoder/LabelEncoder`
- Балансировка классов (например, через `SMOTE`)
- Разделение на обучающую и тестовую выборки

Рисунок А.2 — Страница с информацией о датасете

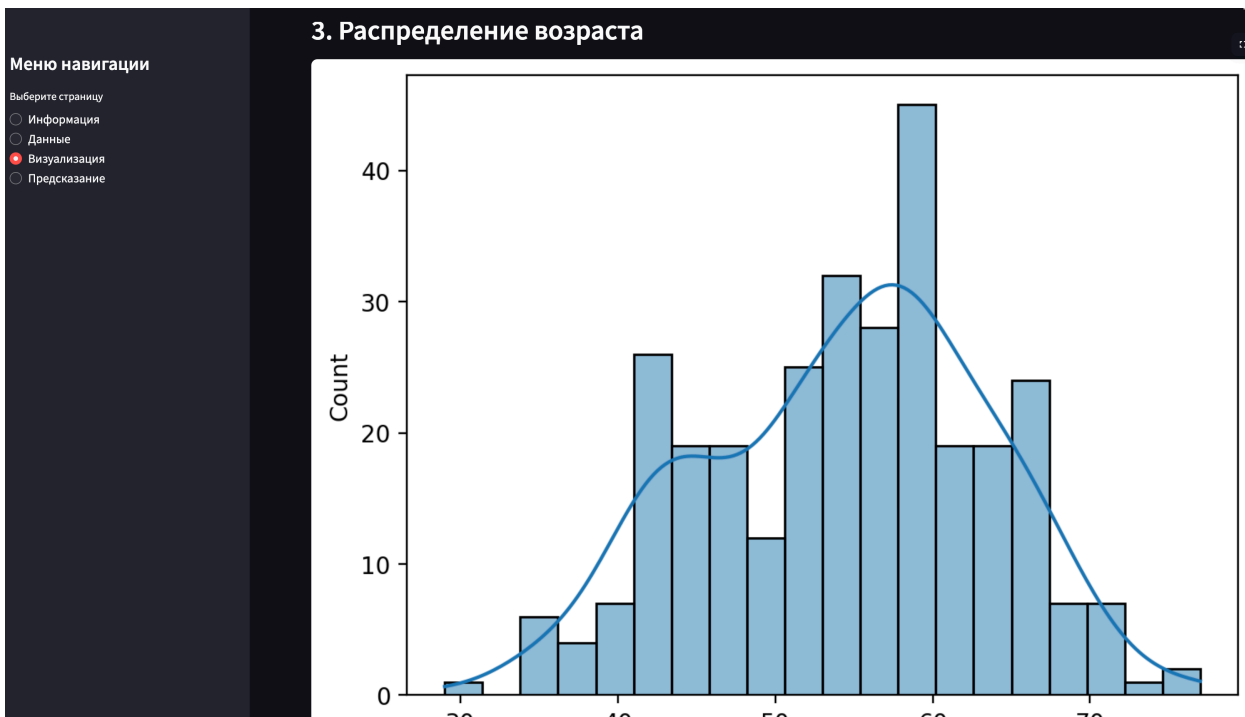


Рисунок А.3 – Страница с визуализацией признаков

Информация

Выберите модель

KNN

1. Загрузка файла с данными

Загрузите CSV-файл для пакетного предсказания

Drag and drop file here

Limit 200MB per file • CSV

Browse files

heart_main.csv

12.5KB

Предсказать из файла

Результаты предсказания

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	num	Предсказание
0	63	1	1	145	233	1	2	150	0	2.3	3	0	6	0	0
1	67	1	4	160	286	0	2	108	1	1.5	2	3	3	2	2
2	67	1	4	120	229	0	2	129	1	2.6	2	2	7	1	1
3	37	1	3	130	250	0	0	187	0	3.5	3	0	3	0	0
4	41	0	2	130	204	0	2	172	0	1.4	1	0	3	0	0
5	56	1	2	120	236	0	0	178	0	0.8	1	0	3	0	0
6	62	0	4	140	268	0	2	160	0	3.6	3	2	3	3	3
7	57	0	4	120	354	0	0	163	1	0.6	1	0	3	0	0
8	63	1	4	130	254	0	2	147	0	1.4	2	1	7	2	2
9	53	1	4	140	203	1	2	155	1	3.1	3	0	7	1	1

Рисунок А.4 – Инференс моделей. Загрузка датасета пользователем

2. Ручной ввод данных

Возраст

41

-

+

Пол

☒ Мужской

☐ Женский

Тип боли

Типичная

Давление (мм рт.ст.)

120

-

+

Холестерин (мг/дл)

200

-

+

Сахар > 120

☐ Да

☒ Нет

ЭКГ

Норма

▼

Макс. пульс

150

-

+

Стенокардия при нагрузке

☒ Да

☐ Нет

Oldpeak

1,00

-

+

Наклон ST

Плоский

▼

Сосуды

0

-

+

Таллий

Норма

▼

Предсказать

Результат

Вероятность отсутствия болезни: 43.43%

Рисунок А.5 – Инференс моделей. Ручной ввод данных всех признаков

ПРИЛОЖЕНИЕ Б

(обязательное)

Исходный код дашборда

Код разработанного приложения

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pickle
from PIL import Image
from tensorflow.keras.models import load_model
from catboost import CatBoostClassifier

@st.cache_data
def load_data():
    return pd.read_csv("heart_main.csv")

@st.cache_resource
def load_models():
    models = {}
    model_files = {
        "KNN": "model_knn.pkl",
        "Gradient Boosting": "model_gb.pkl",
        "CatBoost": "model_catboost.pkl",
        "Random Forest": "model_rf.pkl",
        "Stacking": "model_stacking.pkl"
    }

    for name, file in model_files.items():
        try:
            with open(file, 'rb') as f:
                models[name] = pickle.load(f)
        except Exception as e:
            st.warning(f"Ошибка загрузки {name}: {e}")

    try:
        with open("nn_preprocessor.pkl", "rb") as f:
            nn_preprocessor = pickle.load(f)
        nn_model = load_model("model_nn.h5", compile=False)
        models["Neural Network"] = (nn_preprocessor, nn_model)
    except Exception as e:
        st.warning(f"Ошибка загрузки нейросети: {e}")

    return models

def info_page():
    st.title('Информация')
```

```

st.markdown("ФИО: Колодницкий Илья Михайлович")
st.markdown("Группа: ФИТ-232")

st.image("therock.jpg", width=900, caption="Я на пляже")

st.subheader("Тема РГР: :blue[«Разработка Web-приложения (дашборда) для  
инференса (вывода) моделей ML и анализа данных»] :sunglasses:")

def data_page():
    st.title("Информация о наборе данных")
    data = load_data()

    st.header("Описание предметной области")
    st.write("""
    Цель данного проекта – предсказать наличие сердечного заболевания на
    основе медицинских показателей.
    Сердечно-сосудистые заболевания являются одной из ведущих причин
    смертности во всём мире.
    Этот датасет собран из клинических данных пациентов и используется для
    построения моделей машинного обучения,
    способных выявить возможное заболевание на ранних этапах.
    """)

    st.header("Описание признаков")
    st.write("""
    - **age**: возраст пациента
    - **sex**: пол
    - **cp**: тип боли в груди
    - **trestbps**: артериальное давление в покое
    - **chol**: уровень холестерина
    - **fbs**: уровень сахара натощак
    - **restecg**: результаты ЭКГ
    - **thalach**: максимальный пульс
    - **exang**: стенокардия при физической нагрузке
    - **oldpeak**: депрессия ST
    - **slope**: наклон сегмента ST
    - **ca**: количество крупных сосудов
    - **thal**: результат теста таллия
    - **target (или num)**: наличие сердечного заболевания
    """)

    st.header("Особенности предобработки данных")
    st.write("""
    - Удаление/замена пропущенных значений
    - Нормализация числовых признаков с помощью StandardScaler
    - Кодирование категориальных признаков с помощью
    OneHotEncoder/LabelEncoder
    - Балансировка классов (например, через SMOTE)
    - Разделение на обучающую и тестовую выборки
    """)

    st.header("Разведочный анализ данных (EDA)")

```

```

st.subheader("Распределение целевой переменной")
fig1, ax1 = plt.subplots()
sns.countplot(x='num', data=data, ax=ax1)
ax1.set_title("Наличие сердечного заболевания")
st.pyplot(fig1)

st.subheader("Распределение возраста")
fig2, ax2 = plt.subplots()
sns.histplot(data['age'], bins=20, kde=True, ax=ax2)
ax2.set_title("Возраст пациентов")
st.pyplot(fig2)

st.subheader("Максимальный пульс vs Возраст")
fig3, ax3 = plt.subplots()
sns.scatterplot(x='age', y='thalach', hue='num', data=data, ax=ax3)
ax3.set_title("Связь между возрастом и пульсом")
st.pyplot(fig3)

st.subheader("Корреляционная матрица признаков")
fig4, ax4 = plt.subplots(figsize=(10, 7))
sns.heatmap(data.corr(), annot=True, fmt=".2f", cmap="coolwarm", ax=ax4)
st.pyplot(fig4)

st.header("Пример данных")
st.dataframe(data.head())

```

```

def graphics_page():
    st.title("Визуализация данных")
    data = load_data()

    st.header("1. Распределение целевой переменной")
    fig, ax = plt.subplots()
    sns.countplot(x='num', data=data, ax=ax)
    ax.set_title("Распределение по наличию сердечного заболевания")
    st.pyplot(fig)

    st.header("2. Корреляционная матрица")
    fig, ax = plt.subplots(figsize=(10, 7))
    sns.heatmap(data.corr(), annot=True, fmt=".2f", cmap="coolwarm", ax=ax)
    st.pyplot(fig)

    st.header("3. Распределение возраста")
    fig, ax = plt.subplots()
    sns.histplot(data['age'], bins=20, kde=True, ax=ax)
    st.pyplot(fig)

    st.header("4. Возраст vs Максимальный пульс")
    fig, ax = plt.subplots()
    sns.scatterplot(x='age', y='thalach', hue='num', data=data, ax=ax)
    st.pyplot(fig)

    st.header("5. Boxplot уровня холестерина")
    fig, ax = plt.subplots()

```

```

sns.boxplot(x='num', y='chol', data=data, ax=ax)
st.pyplot(fig)

def predict_page():
    st.title("Предсказание диагноза")
    models = load_models()
    if not models:
        st.error("Модели не загружены")
        return

    model_choice = st.selectbox("Выберите модель", list(models.keys()))

    st.subheader("1. Загрузка файла с данными")
    uploaded_file = st.file_uploader("Загрузите CSV-файл для пакетного
предсказания", type=["csv"])
    if uploaded_file:
        try:
            input_df = pd.read_csv(uploaded_file)

            if model_choice == "CatBoost":
                for col in ['cp', 'fbs', 'restecg', 'exang', 'slope',
'thal']:
                    if col in input_df.columns:
                        input_df[col] = input_df[col].astype(str)

            if st.button("Предсказать из файла"):
                if model_choice == "Neural Network":
                    preprocessor, nn_model = models[model_choice]
                    input_transformed = preprocessor.transform(input_df)
                    prediction = (nn_model.predict(input_transformed) >
0.5).astype(int)
                    input_df["Предсказание"] = prediction
                else:
                    model = models[model_choice]
                    prediction = model.predict(input_df)
                    input_df["Предсказание"] = prediction

            st.subheader("Результаты предсказания")
            st.dataframe(input_df)
            return

        except Exception as e:
            st.error(f"Ошибка при обработке файла: {e}")

    st.subheader("2. Ручной ввод данных")

    col1, col2 = st.columns(2)
    with col1:
        age = st.number_input("Возраст", 1, 120, 50)
        sex = st.radio("Пол", ["Мужской", "Женский"])
        cp = st.selectbox("Тип боли", ["Типичная", "Атипичная",
"Неангинозная", "Бессимптомно"])
        trestbps = st.number_input("Давление (мм рт.ст.)", 80, 200, 120)

```

```

chol = st.number_input("Холестерин (мг/дл)", 100, 600, 200)
fbs = st.radio("Сахар > 120", ["Да", "Нет"])

with col2:
    restecg = st.selectbox("ЭКГ", ["Норма", "ST-T", "Гипертрофия"])
    thalach = st.number_input("Макс. пульс", 60, 220, 150)
    exang = st.radio("Стенокардия при нагрузке", ["Да", "Нет"])
    oldpeak = st.number_input("Oldpeak", 0.0, 10.0, 1.0, 0.1)
    slope = st.selectbox("Наклон ST", ["Восходящий", "Плоский",
"Нисходящий"])
    ca = st.number_input("Сосуды", 0, 3, 0)
    thal = st.selectbox("Таллий", ["Норма", "Фиксированный",
"Обратимый"])

    sex = 1 if sex == "Мужской" else 0
    cp = {"Типичная": 1, "Атипичная": 2, "Неангинозная": 3, "Бессимптомно":
4}[cp]
    fbs = 1 if fbs == "Да" else 0
    restecg = {"Норма": 0, "ST-T": 1, "Гипертрофия": 2}[restecg]
    exang = 1 if exang == "Да" else 0
    slope = {"Восходящий": 1, "Плоский": 2, "Нисходящий": 3}[slope]
    thal = {"Норма": 3, "Фиксированный": 6, "Обратимый": 7}[thal]

    input_data = pd.DataFrame([[age, sex, cp, trestbps, chol, fbs, restecg,
                                thalach, exang, oldpeak, slope, ca, thal]],
                                columns=['age', 'sex', 'cp', 'trestbps', 'chol',
'fbs',
                                'restecg', 'thalach', 'exang', 'oldpeak',
                                'slope', 'ca', 'thal'])

    if st.button("Предсказать"):
        if model_choice == "Neural Network":
            try:
                preprocessor, nn_model = models[model_choice]
                input_transformed = preprocessor.transform(input_data)
                prediction = (nn_model.predict(input_transformed) >
0.5).astype(int)
                proba = [1 - prediction[0][0], prediction[0][0]]
            except Exception as e:
                st.error(f"Ошибка при обработке нейросети: {e}")
            return
        elif model_choice == "CatBoost":
            model = models[model_choice]
            input_df = pd.DataFrame(input_data, columns=[
                'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
                'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal'
            ])
            for col in ['cp', 'fbs', 'restecg', 'exang', 'slope', 'thal']:
                input_df[col] = input_df[col].astype(str)

            prediction = model.predict(input_df)
            proba = model.predict_proba(input_df)[0]
        else:

```

```

        model = models[model_choice]
        prediction = model.predict(input_data)
        proba = model.predict_proba(input_data)[0]

        st.subheader("Результат")
        if prediction[0] == 1:
            st.error(f"Риск заболевания: {proba[1] * 100:.2f}%")
        else:
            st.success(f"Вероятность отсутствия болезни: {proba[0] *
100:.2f}%")

def main():
    st.set_page_config(page_title="Анализ сердца", layout="wide")
    st.sidebar.title("Меню навигации")
    page = st.sidebar.radio("Выберите страницу",
                            ["Информация", "Данные", "Визуализация",
"Предсказание"])

    if page == "Информация":
        info_page()
    elif page == "Данные":
        data_page()
    elif page == "Визуализация":
        graphics_page()
    elif page == "Предсказание":
        predict_page()

if __name__ == "__main__":
    main()

```